Database Design — The Revised Lesson

A corrected, self-contained lesson in entity–relationship (ER) design, from scratch, on paper.

> **About this edition — please read.** This is the corrected "lesson" file you asked for. Two honest notes about how it was made:
>
> 1. The original lesson arrived as a Word (`.docx`) file. In the environment I am running in, I could not open that Word file to edit its exact prose. So this revised lesson is built from the material I *could* read in full — the design primer I wrote for you, plus every example and table from the companion **samples** file (which I read page by page) — with all of the Task-2 corrections applied. It covers the same ground as the original notebook, lesson for lesson, and it is safe to study as your main lesson.
> 2. The original notebook ended with ten practice questions *and their answers*. I could not read those answers, so I could not check them. Instead, this edition ends with a **fresh set of ten practice questions with worked answers that I have verified myself** (Part 13).
>
> Everything below marked **[correction]** is a change made because of the Task-2 review. The full review lives in `database-design-AI-pieces-review.txt`.

How to read this lesson

Four signposts recur, exactly as in the primer:

- **CONTEMPLATE** — stop and think before reading on. No rush.
- **TRY IT** — a small thing to do on paper.
- **KEY IDEA** — the one sentence to remember.
- **PITFALL** — a common mistake, named so you can avoid it.

This whole lesson is paper work. You do not touch a computer here. The touching-a-computer work is in `database-design-practice-lesson.txt`.

A note on the running examples (read once)

To keep things concrete, this lesson leans on a few small worlds and returns to them again and again, so the same rows reappear across sections and across the companion **samples** file. The main ones:

- a **small library** — `member`, `book`, `copy`, `loan` (the big worked example in Part 10);
- a **school** — `group`, `student` (the simplest one-to-many);
- a **shop** — `orders`, `product`, `order_line` (the many-to-many bridge and the frozen price);

- a few one-off shapes — a `person` / `passport` pair (one-to-one), a `category` tree (self-reference), a `prescription` (three-way fact), a building's `room` (weak entity).

Wherever a table appears here, the samples file shows the *same* table with the *same* rows, plus its diagram. Read the two side by side.

Part 1 — What "designing a database" actually means

Before any detail, the big picture. **Designing a database means deciding what things you will keep, what facts you will keep about each, and how those things connect — all before you store a single row.** The design is a blueprint. The rows that flow in later are the data. A good blueprint makes the data behave; a poor one fights you forever.

A deeper way to put the same thing: **a database is a deliberate *model* of a slice of the real world.** The world is endlessly detailed; your model keeps only the facts your program needs and throws the rest away. A library's database "knows" a book has a title and an author, but not that its cover is scuffed or smells of coffee. Design is the craft of choosing what to keep — and *entity– relationship* (ER) modelling, the subject of this lesson, is just the disciplined way to draw that choice.

Designers work at three levels, from fuzzy to concrete:

- **Conceptual** — the things and their connections, in plain language. "We have students, groups, parameters, and scores; a score is about a student and a parameter." No tables yet. This is where most of the thinking happens.
- **Logical** — the same picture turned into tables, columns, keys, and foreign keys, but not tied to any particular database product.
- **Physical** — the real `CREATE TABLE` statements for a specific database (Postgres, SQLite, …), with exact types and indexes.

KEY IDEA: design top-down. Get the conceptual picture right on paper first; the logical and physical levels are mostly careful translation after that.

The same idea at all three levels. Take one small fact — "a student belongs to a group" — and watch it harden as you move down the levels:

Level	How "a student belongs to a group" looks
Conceptual	A sentence: *"Each student is in exactly one group; a group holds many students."* No tables yet.
Logical	Two tables, a foreign key: `group(group_id, name)` and `student(student_id, full_name, group_id → group)`.
Physical (Postgres)	`CREATE TABLE "groups" (group_id uuid PRIMARY KEY, name text NOT NULL);` and a `student` table whose `group_id uuid NOT NULL REFERENCES "groups"(group_id)`.

Notice how nothing new is *invented* going down — the physical level is the conceptual sentence, made exact. (Notice too that `group` became `groups`: a reserved-word fix we meet properly in Part 5.)

A design is **good** when it is:

- **faithful** — it matches how the real world actually behaves;
- **non-redundant** — each fact is stored once, in one place;
- **answerable** — it can answer the questions you need to ask of it;
- **simple** — no more tables or columns than the job requires.

Here is each quality with a one-line "bad vs good" so the words mean something:

Quality	A design that fails it	The same design, fixed
faithful	one `group_id` column, though a student really belongs to several groups	a bridge table, so a student can join many groups
non-redundant	the group's name copied onto every student row	the name kept once in `group`, students hold only `group_id`
answerable	storing a loan's data but no borrow date, so "what's overdue?" cannot be asked	a `due_on` date on every loan
simple	separate `phone1`, `phone2`, `phone3` columns "just in case"	one `phone` table that holds as many as needed

CONTEMPLATE: of those four, "answerable" is the quiet one. Before designing anything, write down the *questions* and *actions* the system must support — "who scored below 50 last term?", "add a student to a group", "show a group's average over time". A design that cannot answer its own purpose questions is wrong no matter how tidy it looks. We will return to this list at the very end (Part 9, step 10) to check our work against it.

A worked questions/actions list. For the little library we design in Part 10, the list looks like this. Keep it; you will test the finished design against it row by row:

#	A question to answer, or an action to perform	Touches
1	Who has book "Clean Code" out right now?	loan, copy, book, member
2	Which loans are overdue (due date passed, not returned)?	loan
3	How many loans has Ada had, ever?	loan, member
4	Add a new member.	member
5	Lend a copy to a member.	loan
6	Record that a copy came back.	loan

The point is not the list's neatness; it is that every one of these must be possible against the final tables, or the design has quietly failed.

Part 2 — Entities: the "things" (type vs instance)

An **entity** is a thing you keep information about: a student, a book, a group, a score. Each entity type becomes a **table**. The single most important distinction for a beginner:

- an entity **type** is the *kind* of thing — it becomes a **table** (the cookie-cutter);
- an entity **instance** is *one* of them — it becomes a **row** (one cookie).

So "student" the table is the kind; each enrolled person is a row.

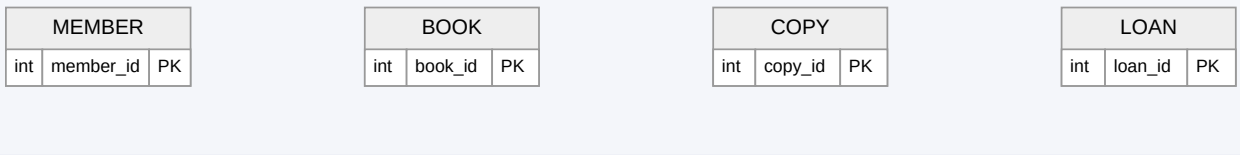
Entity *type* (the table)	Some *instances* (the rows)
`member`	Ada Lovelace; Alan Turing; Grace Hopper
`book`	*The Pragmatic Programmer*; *Clean Code*
`group`	Morning; Evening

How to find entities: write a sentence or two describing your world, then **underline the nouns.** Most strong, repeated nouns are entities; the facts hanging off them are attributes (Part 3); the verbs between them are relationships (Part 4). "A **student** belongs to a **group** and receives a **score** on each **parameter** every **period**" → five candidate entities.

A worked underline. Take the library sentence: "*A member borrows a copy of a book, and each loan has a due date.*" Sort every noun:

Noun	Verdict	Why
member	**entity**	we keep facts about it (name, email) and many rows point at it
copy	**entity**	a physical item with its own shelf code and history
book	**entity**	the title/author/ISBN, shared by many copies
loan	**entity** (a relationship that earned a table)	it has its *own* facts — the dates
due date	**attribute**	a single value belonging to one loan

The five surviving entities, drawn as bare boxes before any lines are added:



PITFALL: not every noun is an entity. Some are just attributes ("name", "colour"), and some are relationships in disguise ("enrolment", "loan"). The test, used again and again: *do we keep facts about this thing, or will many rows share it?* If yes, it earns its own table; if it is a single plain value belonging to one row, it is an attribute.

TRY IT: the same word can switch roles depending on the world. In a corner shop, `colour` is a plain value on a product (an attribute). In a paint factory, colours have codes, bases, and suppliers that many products share — now `colour` is an entity with its own table. Nothing about the word decides this; the *purpose* does. (This is Q1 in Part 13.)

Part 3 — Attributes: the facts about each thing

An **attribute** is one fact kept about an entity; it becomes a **column**. Attributes come in kinds you must recognise:

- **atomic** vs **composite** — a composite attribute has parts worth splitting. "Full name" may split into given/family name; an address splits into street/city/postcode. Split when you will query or sort by the parts.
- **single-valued** vs **multi-valued** — a person has one birth date (single), but possibly many phone numbers (multi). **A multi-valued attribute is never many columns and never a list in one cell — it becomes its own table with a foreign key back** (Part 7).
- **stored** vs **derived** — a derived value can be computed from others. Store `birth_date`, show *age*. Store the line items, compute the *total*. The rule is **derive, don't store**, because a stored copy drifts out of step with what it came from.

The kinds at a glance, each with a concrete reading:

Kind	Example	Rule of thumb
atomic	`email`, `due_on`	leave it as one column
composite	`full_name` → `given_name` + `family_name`	split it if you'll sort/search the parts
single-valued	`born_on`	one column
multi-valued	a member's many phone numbers	its own table, FK back (never `phone1,2,3`)
stored	`born_on`, the line items of an order	keep
derived	`age`, an order's total	don't store — recompute

Composite — before and after. A `member` that crams the whole name into one cell cannot be sorted by family name:

member_id	full_name
1	Ada Lovelace
2	Alan Turing

Split it, and now "list members by family name" is trivial:

member_id	given_name	family_name
1	Ada	Lovelace
2	Alan	Turing

Multi-valued — never columns, never a list. A member with several phones is the classic trap. The wrong fixes are `phone1, phone2, phone3` (caps the count, and "who has 555-2222?" must search three columns) or `"555-1111, 555-2222"` in one cell (which you cannot search or count). The right shape is a second table, one phone per row (we meet this again in Part 8 as 1NF):

`member`:

member_id	full_name
1	Ada Lovelace

`phone` (FK back to member, as many rows as needed):

phone_id	member_id	number
1	1	555-1111
2	1	555-2222

[correction] The frozen-fact exception. There is one important exception the samples file makes well and the primer only hinted at: a value you capture **on purpose, at a moment in time, because it must never change afterward** is *not* a derived value you should drop — it is a **frozen fact**. The classic case is `unit_price_at_sale` on an order line: the product's price today is irrelevant; the order must remember what the customer actually paid. Keep it. It only *looks* like redundancy.

Derive vs freeze — four quick calls:

Value	Verdict	Why
an order's total	**derive**	just the sum of its lines; recompute it
a student's age	**derive**	compute from `born_on` and today
`unit_price_at_sale`	**freeze (store)**	the price paid *then* must not move
a scored period in student-panel	**freeze (store)**	once scored, the number stands

KEY IDEA: derive values you can always recompute; freeze values that record "what was true at the time". In student-panel, a recorded score is a frozen fact of its period — once scored, the number stands even if the parameter is later reconfigured.

Part 4 — Relationships, cardinality, and participation

A **relationship** is how two entities connect — usually a verb. "A student *belongs to* a group." "A member *borrows* a copy."

Degree — how many entity *types* a relationship joins. Name it before the numbers:

- **unary** (a thing linked to *more of its own kind*) — a group inside a group, an employee who manages another employee; drawn as one box with a line looping back to itself.
- **binary** (two entity types) — by far the most common: *group–student*, *member–loan*.
- **ternary** (three at once) — "a doctor prescribes a drug to a patient" ties three things in a single fact.

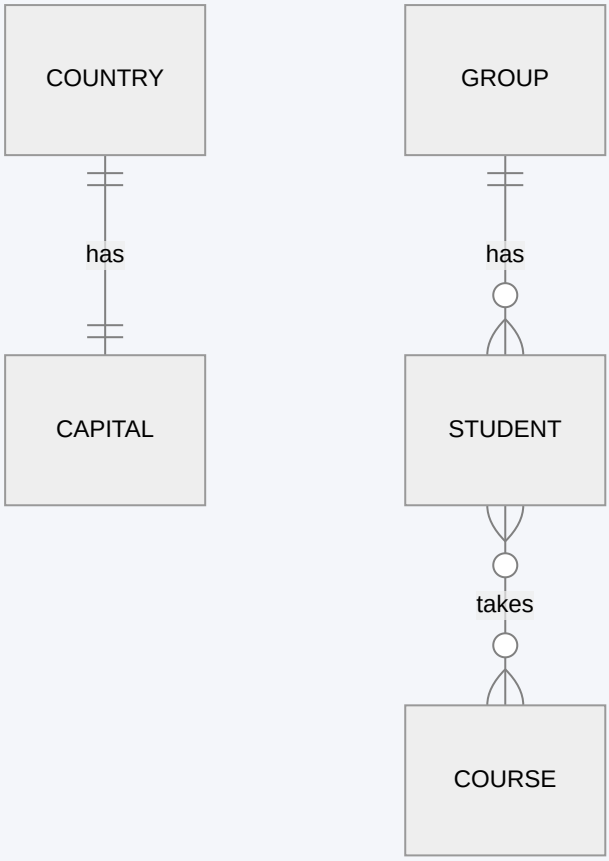
Most relationships are binary; the unary and ternary cases are two of the five hard patterns, so we meet them again in Part 7 (self-reference and the three-way fact).

Beyond its degree, two measurements make a relationship precise, and you must state **both**:

Cardinality — how many on each side:

- **one-to-one (1:1)** — one country has one capital.
- **one-to-many (1:N)** — one group has many students; each student is in one group.
- **many-to-many (M:N)** — a student takes many courses; a course has many students. (M:N always needs a table in the middle — Part 7.)

Each shape as a picture (and the sentence it reads as):



- `COUNTRY ||--|| CAPITAL` — one country has exactly one capital, and back.
- `GROUP ||--o{ STUDENT` — one group has zero-or-many students; each student has exactly one group.
- `STUDENT }o--o{ COURSE` — many-to-many; this one cannot stand as a plain line and will become a bridge table in Part 7.

Participation — whether a side is **optional** (minimum zero) or **mandatory** (minimum one). "Must every loan have a member? Yes (mandatory). Must every member have a loan? No (optional — a brand-new member has none)." The optional side is exactly where a **NULL** or a *zero-count* is allowed and meaningful.

Relationship	One side	Other side
group ↔ student	a group may have **zero** students (optional)	a student must have **one** group (mandatory)
member ↔ loan	a member may have **zero** loans (optional)	a loan must have **one** member (mandatory)
person ↔ passport	a person may have **zero or one** passport (optional)	a passport must have **one** person (mandatory)

TRY IT: for any two entities, say the pairing aloud **in both directions** with a number and an optionality each time: "one group has *zero or many* students; one student has *exactly one* group." If you can say both sentences, you have the relationship fully specified.

A filled-in "say it both ways" worktable — do this for every line you draw:

Entities	One … has … of the other	… and the other has …
group, student	one group has zero-or-many students	one student has exactly-one group
book, copy	one book has one-or-many copies	one copy is of exactly-one book
member, loan	one member has zero-or-many loans	one loan has exactly-one member

KEY IDEA: every relationship = a direction + a number (one/many) + an optionality (zero/one minimum), stated **both ways**. Skip either and the design is ambiguous.

Part 5 — Keys: how a row is identified and how tables point at each other

A **key** is a column (or set of columns) that identifies a row.

- **candidate key** — any column(s) that could uniquely identify a row.
- **primary key (PK)** — the one candidate key you choose as *the* identifier.
- **natural key** — a real-world identifier (email, ISBN, national id).
- **surrogate key** — an id invented by the system purely to be a stable handle.

The four kinds on one small `book`, so you can see them coexist:

book_id (surrogate PK)	title	author	isbn (natural, UNIQUE)
1	The Pragmatic Programmer	Hunt & Thomas	9780135957059
2	Clean Code	R. C. Martin	9780132350884
3	Structure and Interpretation	Abelson & Sussman	9780262510875

Both `book_id` and `isbn` are *candidate* keys (each is unique). We pick `book_id` as the **primary** key because it never changes, never repeats, and is never empty (a primary key must always hold a value — never NULL); `isbn` stays as a **natural** key under a `UNIQUE` rule so you can still look a book up by its real-world identifier.

Natural keys change and sometimes repeat; surrogate keys never do. **Modern practice: give each table a surrogate primary key, and *also* keep the useful natural value under a `UNIQUE` rule** so it still cannot duplicate.

[correction] Two kinds of surrogate key — INT and UUID. This is the one real addition the Task-2 review called for. A surrogate key can be:

- an **auto-numbered integer** (1, 2, 3, …) — small, readable, perfect for learning. The samples file uses these everywhere.
- a **UUID** — a long random id like `7f3a1c2e-…-9c2b`. Better when ids must be unique across many machines, or must not reveal how many rows exist or in what order they were created.

Both are surrogate keys; neither is "more correct". **student-panel uses UUID primary keys plus a unique human `student_code`.** So when you see `1, 2, 3` in the samples and long random ids in student-panel, that is the *same decision in two costumes* — not a conflict.

The same two costumes side by side — identical design, different handle:

Style	PK column	A sample value	Also keep
INT (learning)	`student_id`	`1`	`student_code` UNIQUE
UUID (student-panel)	`id`	`7f3a1c2e-9b55-4a0e-8d21-0c2b9c2b7a10`	`student_code` UNIQUE

Continuing with keys:

- **composite key** — a key made of more than one column (e.g. a bridge table keyed on `(student_id, course_id)`).
- **foreign key (FK)** — a column that points at another table's primary key. This is how tables connect. The FK carries a promise the database enforces: "this pointer must point at a row that really exists" — called **referential integrity**. A row that points at something gone is an **orphan**, and the FK refuses to create one.

Where the FK goes: in a one-to-many, **the foreign key lives on the MANY side.** One group, many students → the *student* row carries `group_id`, not the other way around. The child points at the parent. Seen in rows — the pointer is on `student`, and three students happily share group `1`:

`group`:

group_id	name
1	Morning
2	Evening

`student` (FK `group_id` on the many side):

student_id	full_name	group_id
1	Ada	1
2	Alan	1
3	Grace	2

For a 1:1, put the FK on the side that is **never empty**, and mark it `UNIQUE NOT NULL` (one passport belongs to exactly one person → put `person_id` on the passport row). The `UNIQUE` is what makes it 1:1 rather than 1:N — it forbids a second passport pointing at the same person:

`person`:

person_id	full_name
1	Ada
2	Alan

`passport` (FK `person_id`, UNIQUE so no person gets two):

passport_id	person_id (UNIQUE)	number
1	1	X1234567

ON DELETE rules — when a parent row is deleted, each FK decides what happens:

- **RESTRICT** — refuse the delete while children exist (protect history).
- **CASCADE** — delete the children too (they cannot exist without the parent).
- **SET NULL** — keep the children but blank the pointer (the link was optional).

A good habit from the samples file: **say each delete rule as a sentence** — "you may not delete a member who still has loans" (RESTRICT), "deleting an order deletes its order lines" (CASCADE).

Rule	The sentence	Reach for it when
RESTRICT	"you may not delete a member who still has loans"	history must be protected; block the delete
CASCADE	"deleting an order deletes its order lines too"	the child has no life without the parent
SET NULL	"deleting a teacher leaves the class with no teacher"	the link is optional and may just empty out

[correction] Reserved words. At the physical level, some words already mean something to the database — `order`, `group`, `user` are reserved in Postgres (which student-panel uses). Do not name tables or columns with them directly; use `orders`, `groups`, `app_user`, or quote them. The primer never mentioned this; the samples file flags it, and it matters.

Reserved word	Safe name to use
`order`	`orders`
`group`	`groups`
`user`	`app_user`

Part 6 — Reading and drawing crow's-foot diagrams

A relationship is drawn as a line between two tables. Near each end are up to two marks: the mark **closest** to the table is the **maximum** (one, or many); the mark just before it is the **minimum** (zero = optional, or one = mandatory).

Symbol	Reads as
`|` (bar)	one
`o` (circle)	zero / optional
`<` (crow's foot)	many
`||`	one and only one (min 1, max 1)
`|<`	one or many (min 1, max many)
`o<`	zero or many (min 0, max many)
`o|`	zero or one (min 0, max 1)

Say the two marks in order, far-then-near: the far mark is the **minimum**, the near mark is the **maximum**. "`o<` next to a table" = minimum zero, maximum many = "zero or many of it".

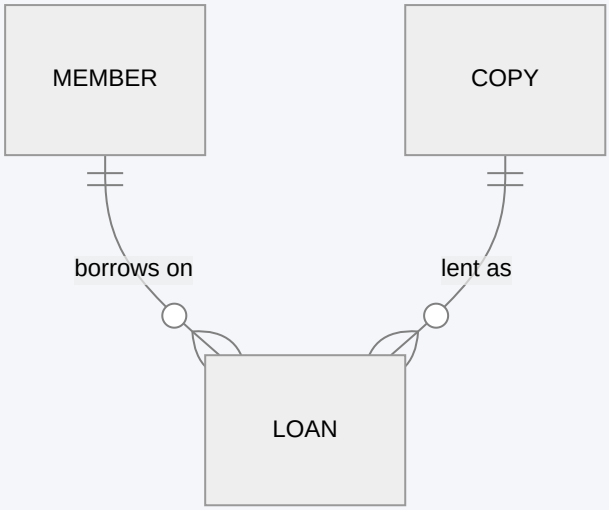
[correction] The mirroring. Read the marks from the **outside in** toward each table. Because the two ends face the line from opposite sides, the *same* meaning is drawn mirrored: "zero or many" is `o<` on the left end and `>o` on the right end. They are identical — do not read `>o` and `o<` as two different ideas. The primer used this convention without explaining it; here it is explicit.

A worked reading — the library line:

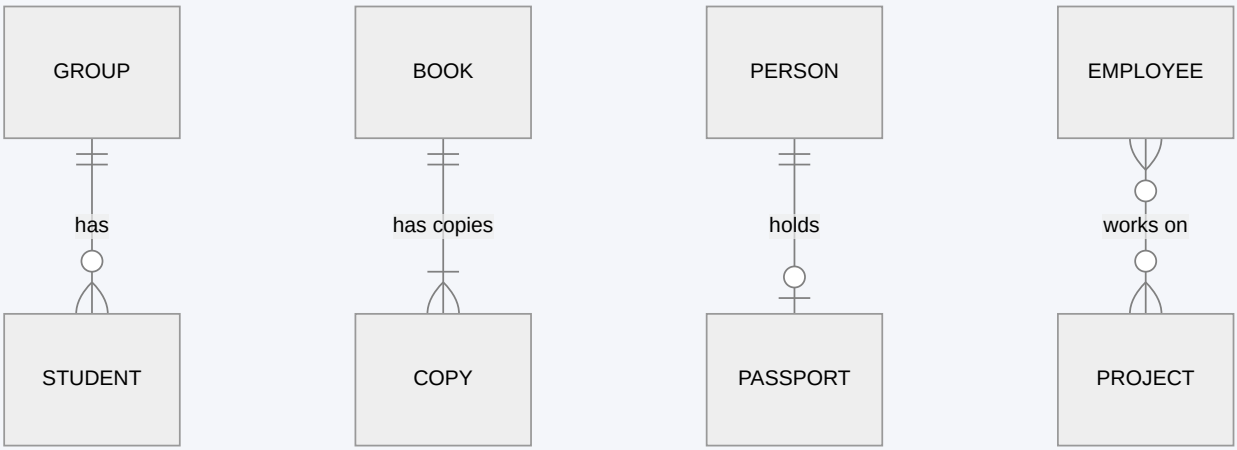
```
Member  ||-----o<  Loan  >o-----||  Copy
```

- Member side: `||` next to Member = exactly one member; `o<` next to Loan = zero or many loans → "one member has zero or many loans; each loan belongs to exactly one member."
- Copy side: `>o` next to Loan (mirrored `o<`) = zero or many loans; `||` next to Copy = exactly one copy → "one copy has zero or many loans; each loan is of exactly one copy."

The same picture as a Mermaid diagram (renders when converted with a Mermaid-capable tool):



A small gallery, each line with the sentence it reads as (cover the right column and practise):



Mermaid line	Read it as
`GROUP ||--o{ STUDENT`	one group has zero-or-many students; each student has exactly one group
`BOOK ||--|{ COPY`	one book has one-or-many copies; each copy is of exactly one book
`PERSON ||--o| PASSPORT`	one person holds zero-or-one passport; each passport is of exactly one person
`EMPLOYEE }o--o{ PROJECT`	many-to-many (both optional) — needs a bridge

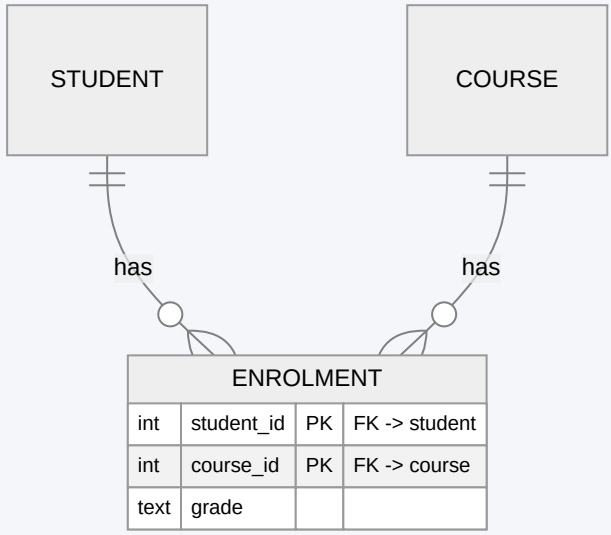
Note `BOOK ||--|{ COPY` uses `|{` (one-or-many): a book exists in the catalogue only once we own at least one copy of it. That one mandatory mark is a real modelling decision, not decoration.

TRY IT: cover the captions in the samples file, read the symbols aloud as full sentences, and check your sentence against the caption. If they match, you can read crow's-foot.

Part 7 — The five hard patterns (and the one move behind three of them)

Most designs are easy until you hit one of five shapes. Learn to recognise them.

7.1 Many-to-many → a bridge table. A student takes many courses; a course has many students. You cannot express M:N with a single FK. Put a **bridge** (junction/associative) table in the middle, carrying one FK to each side:



Any fact *about the pairing* (a grade, a date) lives **on the bridge**, not on either side.

In rows — `student` and `course` on the two sides, `enrolment` in the middle holding the pairing *plus* its own fact, the grade:

`student`:

student_id	full_name
1	Ada
2	Alan

`course`:

course_id	title
10	Logic
20	Numerical Methods

`enrolment` (bridge; the pair is the key, `grade` is a fact of the pairing):

student_id	course_id	grade
1	10	A
1	20	B
2	10	A

The pair `(1, 10)` can appear only once, so Ada cannot be enrolled in Logic twice — the composite key does that work for free.

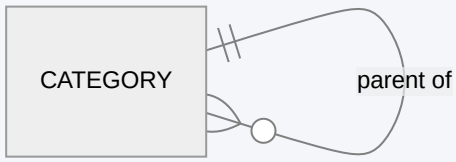
[correction] Choosing the bridge's key. Two valid styles:

- **pair as the key** — `PRIMARY KEY (student_id, course_id)`. Use this when the pairing is naturally unique and nothing else needs to point at the bridge row.
- **a surrogate id on the bridge** — `enrolment_id` as PK, **plus a `UNIQUE` constraint on the pair**. Use this when some *other* table must reference an individual bridge row. If you take this route, do not forget the `UNIQUE` on the pair, or you silently allow duplicate pairings.

Decide on purpose; both are correct. (The samples file shows both; this note makes the "when" explicit.)

7.2 One-to-one → usually merge. If two tables are 1:1, ask whether they should just be one table. Keep them separate only for a reason: optional data kept apart, security, or very wide rows. When separate, FK on the never-empty side, `UNIQUE NOT NULL`. The passport rows from Part 5 are the "kept separate" case; most of the time you would simply add a `passport_number` column to `person` and be done.

7.3 Self-reference (a tree). A thing that contains more of its own kind — a group inside a group, a category with sub-categories, an employee with a manager. Add **one column that points back at the same table** (`parent_id`), `NULL` at the root. One column handles any depth; the cost is that "all descendants" needs a recursive query. Do **not** use `parent` , `grandparent` , `great_grandparent` columns — that caps the depth and repeats the "repeating columns" mistake.



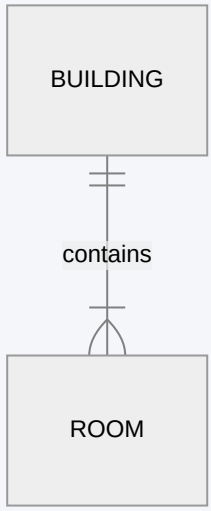
In rows — one table, the `parent_id` pointing back into it; `NULL` marks a root:

category_id	name	parent_id
1	Books	*(null)*
2	Fiction	1
3	Science	1
4	Physics	3

Books is a root (no parent); Physics sits under Science, which sits under Books — three levels deep with a single column, and a fourth level would need no schema change at all.

student-panel uses this twice — groups form a tree and parameters form a tree, each with a self-referencing parent and `NULL` at the root.

7.4 Weak / dependent entity. Something that cannot exist without its parent and is identified partly by it — an order *line* only means something inside its order. Its key often includes the parent's key, and its delete rule is usually CASCADE. A building's `room` is the same shape — "room 101" means nothing until you say *which building*:



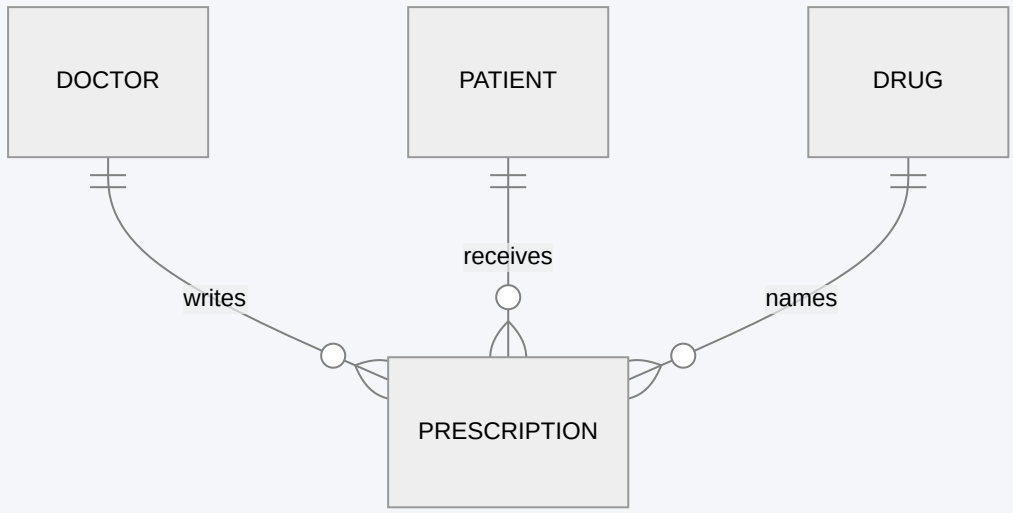
`room` (identified by its building **plus** its own number):

building_id	room_no	capacity
1	101	30
1	102	25
2	101	40

Key = `(building_id, room_no)`. Room 101 appears twice — once per building — and that is correct, because the building is part of the room's identity.

7.5 Ternary (a three-way fact). Sometimes a fact genuinely joins **three** things at once: a *prescription* ties a doctor, a drug, and a patient; a *supply* ties a supplier, a part, and a

project. Model it as a table in the middle holding all three FKs.



In rows — one middle table carrying three pointers plus the date it happened:

prescription_id	doctor_id	patient_id	drug_id	prescribed_on
1	7	1	3	2024-03-01
2	7	1	3	2024-06-01

[correction] The UNIQUE-on-the-triple note. If you give such a table its own surrogate id (`prescription_id`), you must still **decide whether the natural combination should be `UNIQUE`** — exactly the decision the samples file states for the order-line bridge but leaves unsaid for the prescription. Sometimes the triple *should* be unique; sometimes the real world allows repeats (the same doctor may prescribe the same drug to the same patient again on a later date, in which case a date belongs in the key and the triple is deliberately *not* unique). Make the choice consciously and write it down.

In the rows above the triple `(7, 1, 3)` appears **twice** — rows 1 and 2 differ only by date — so here you would *not* put `UNIQUE(doctor_id, patient_id, drug_id)`; the date is part of what makes each prescription distinct. If repeats were forbidden, you *would* add that UNIQUE. Same decision as `order_line`.

The one move. Notice that 7.1 (many-to-many), "a link that has its own facts", and 7.5 (a three-way fact) are the **same move**: when a relationship is many-to-many, *or* carries its own attributes, *or* joins more than two things, it stops being a line and **becomes a table**. The **disguise test** for a ternary (from the samples file): drop one of the three nouns — if the remaining sentence is still a complete, real fact, it was not truly a three-way fact.

Applying the disguise test to the prescription: "Doctor D prescribes drug G" — to *whom*? Incomplete. "Patient P is given drug G" — on whose *order*? Incomplete. Every noun is load-bearing, so it is a true ternary.

KEY IDEA: if a relationship is M:N, has its own facts, or joins 3+ things, it wants to *be* a table, not a line. In student-panel, the **scores/evaluations** table is exactly this — a table in the middle linking a student and a parameter (for a period) and carrying the score. The thing that looks like "just a link" is the most important table in the design.

Part 8 — Normalisation: removing the anomalies

Normalisation is a set of checks that remove duplication. Duplication causes three **anomalies**:

- **update anomaly** — a fact copied onto many rows drifts apart when one copy changes (rename a member whose name was copied onto every loan, and the loans disagree).
- **insert anomaly** — you cannot record a thing because there is nowhere to put it (a member who has not borrowed anything yet has no row in a loan-only table).
- **delete anomaly** — removing one row destroys an unrelated fact (deleting a member's last loan erases the member entirely).

See all three in one bad table — `loan` with the member's facts copied onto it:

loan_id	member_id	member_name	member_email	book_title
1	1	Ada Lovelace	ada@example.com	Clean Code
2	1	Ada Lovlace	ada@example.com	The Pragmatic Programmer
3	2	Alan Turing	alan@example.com	Clean Code

- **update anomaly:** someone fixed Ada's name on loan 1 but not loan 2 (note the typo "Lovlace" that still lingers) — now the database disagrees with itself.
- **insert anomaly:** Grace, a new member who has borrowed nothing, cannot be recorded at all — there is no loan row to hang her on.
- **delete anomaly:** delete Alan's only loan (row 3) and Alan vanishes from the database entirely, though he is still a member.

The cure for all three is the same: give the member its own table and let `loan` hold only `member_id`. We do exactly that in 3NF below.

[correction] What "depends on" means (a beginner-friendly definition from the samples file): *A depends on B when knowing B tells you A.* If you know the `member_id`, you know the member's name — the name **depends on** the member id. This single idea powers the normal forms.

The three forms you need, with the classic mnemonic — *every non-key column must depend on the key, the whole key, and nothing but the key:*

- **1NF — the key.** One value per cell; no repeating columns (`phone1` , `phone2` , `phone3`) and no lists stuffed in a cell. Repeating things get their own table (Part 7). A list in a cell cannot be searched or counted.

- **2NF — the whole key.** With a composite key, no column may depend on only *part* of it. On `order_line(order_id, product_id)`, `product_name` depends on `product_id` alone → it belongs in a `product` table, not on the line.
- **3NF — nothing but the key.** No column may depend on another non-key column. If `employee` stores `dept_id` *and* `dept_name`, the name depends on `dept_id`, not on the employee → move it to a `dept` table.

1NF worked — one value per cell. Bad: a list of phones crammed into a cell. You cannot cleanly ask "who has 555-2222?":

member_id	full_name	phones
1	Ada	555-1111, 555-2222

Fixed: one phone per row in its own table (the multi-valued move from Part 3):

`member`:

member_id	full_name
1	Ada

`phone`:

phone_id	member_id	number
1	1	555-1111
2	1	555-2222

2NF worked — the whole key. Bad: `order_line` keyed on `(order_id, product_id)`, but carrying `product_name`, which depends on `product_id` *alone* (only part of the key). The name repeats on every line that mentions the product:

order_id	product_id	product_name	quantity
1	1	Blue pen	3
2	1	Blue pen	1

Fixed: `product_name` moves to `product`, where it depends on the whole key `product_id`. The line keeps only what depends on the full pair:

`product`:

product_id	name	list_price
1	Blue pen	1.50
2	Red pen	1.50

`order_line`:

order_id	product_id	quantity	unit_price_at_sale
1	1	3	1.50
2	1	1	1.50

3NF worked — nothing but the key. Bad: `student` carrying its group's name. `group_name` depends on `group_id` (another non-key column), not on `student_id`. Change "Morning" once and you must change it on every student:

student_id	full_name	group_id	group_name
1	Ada	1	Morning
2	Alan	1	Morning
3	Grace	2	Evening

Fixed: the name lives once in `group`; `student` keeps only the FK `group_id`:

`group`:

group_id	name
1	Morning
2	Evening

`student`:

student_id	full_name	group_id
1	Ada	1
2	Alan	1
3	Grace	2

Deliberate denormalisation. Sometimes you store a redundant value on purpose for speed, accepting the upkeep cost — a conscious trade, not an accident.

[correction] Not a break: the frozen fact. `unit_price_at_sale` on an order line looks like it violates normalisation, but it does not: it is a historical value captured at sale time, not a copy of the product's current price. Keeping it is correct (see Part 3). Recognising the difference between a *drifting copy* (bad) and a *frozen record* (good) is a real design skill.

The tell: ask "*if the source changes, must this value change too?*" For `group_name` copied onto a student, yes — so it is a drifting copy, and 3NF removes it. For `unit_price_at_sale`, no — the past price must *not* move — so it is a frozen fact, and you keep it. Same shape on the page, opposite verdicts.

Part 9 — The design method (one merged ten-step routine)

[correction] Unifying the two lists. You have seen two ten-step routines: the primer's Part 7, and the samples file's Lesson 15. They are the *same method* sliced differently, and the samples file itself invites you to compare them. Here is a single merged routine that keeps the best of both — it keeps the primer's explicit **"draw it"** step, and it promotes the samples file's **"questions/actions"** and **"delete rules"** to steps of their own so a beginner cannot skip them:

1. **State the purpose** — one or two sentences on what the system is for.
2. **List the questions and actions** it must support (the "answerable" test).
3. **Find the entities** — underline the nouns; each becomes a table.
4. **Find the attributes** — mark composite / multi-valued / derived.
5. **Choose keys** — a surrogate PK (INT or UUID), plus `UNIQUE` on natural keys.
6. **Find the relationships** — state cardinality **both ways** + participation ("say the pairs aloud").
7. **Resolve the hard patterns** — M:N → bridge, ternary → middle table, self-reference → `parent_id`, weak entity, 1:1.
8. **Place the foreign keys** — FK on the many side; 1:1 on the never-empty side.
9. **Choose the delete rules** — RESTRICT / CASCADE / SET NULL, said as sentences.
10. **Draw it** — a crow's-foot diagram, so the shape is visible.
11. **Normalise** — check 1NF, 2NF, 3NF; remove the anomalies.
12. **Map to tables** — write the `CREATE TABLE`s.
13. **Test with rows** — put a few real rows in and answer the Step-2 questions.

(That is the two lists merged; both the primer and the samples file "arrive" at this same place — nothing is lost, and the two skippable steps are now explicit.)

Each step, drawn. To make the routine concrete, here is one small world — *a gym* — carried through all thirteen steps. (The samples file works the same gym in Lesson 18, so you can read them together.)

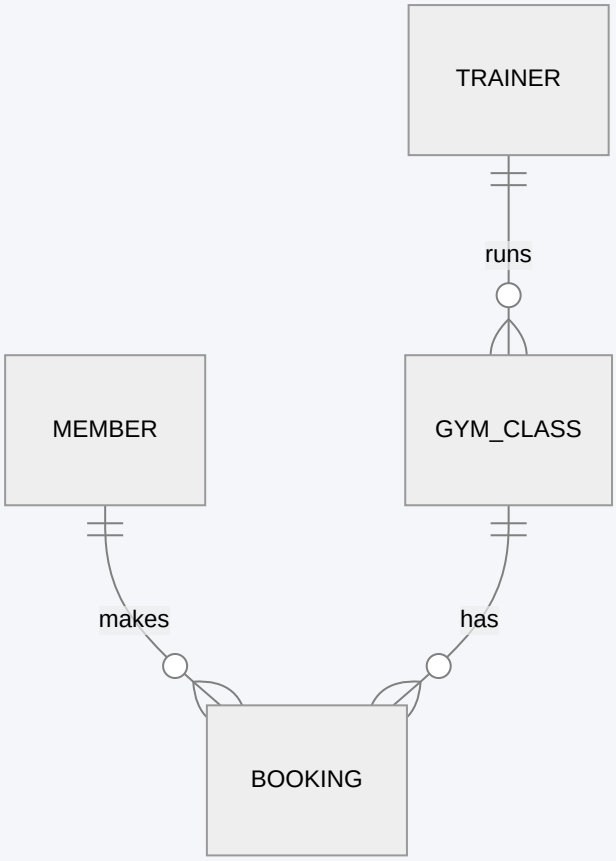
1. **Purpose.** "Members join the gym. Trainers run classes. A class happens at a time and has a capacity. Members book a place in a class."
2. **Questions/actions.** Who is booked into tonight's spin class? How many places are left? Add a member; schedule a class; book a place.
3. **Entities.** Underline the nouns → `member`, `trainer`, `gym_class`, and a booking (a member taking a place in a class).

Noun	Verdict
member	entity
trainer	entity
class	entity (`gym_class` ; `class` is risky, so name it fully)
booking	entity (an M:N link with its own fact)

4. **Attributes.** member: full_name, email, joined_on. trainer: full_name. gym_class: title, starts_at, capacity. booking: booked_on. Mark the derived one now: *places left* = capacity − bookings → **derive, don't store**.
5. **Keys.** A surrogate PK each (`member_id` , `trainer_id` , `class_id`); keep `member.email` `UNIQUE` as a natural key.

Table	PK	Natural key kept UNIQUE
member	member_id	email
trainer	trainer_id	—
gym_class	class_id	—

6. **Relationships, both ways.** One trainer runs zero-or-many classes; each class has exactly one trainer. One member makes zero-or-many bookings; one class has zero-or-many bookings; each booking is of exactly one member and one class.
7. **Resolve the hard patterns.** "A member books many classes; a class has many members" is M:N → the `booking` bridge resolves it, carrying its own fact `booked_on` . No ternary, no tree here.
8. **Place the FKs.** `gym_class.trainer_id` → trainer (many side). `booking` holds `member_id` → member and `class_id` → gym_class (the bridge carries one FK to each side).
9. **Delete rules, as sentences.** "You may not delete a trainer who still runs classes" (RESTRICT on `gym_class.trainer_id`). "Deleting a class deletes its bookings" (CASCADE on `booking.class_id`) — a booking has no life without its class.
10. **Draw it.**



1. **Normalise.** Each fact sits once. `places_left` is *not* stored (it's derived). The `(member_id, class_id)` key stops a member booking the same class twice. No repeating columns, no partial or transitive dependencies.
2. **Map to tables.**

```
member(member_id PK, full_name, email UNIQUE, joined_on)
trainer(trainer_id PK, full_name)
gym_class(class_id PK, title, starts_at, capacity, trainer_id FK -> trainer)
booking(member_id FK -> member, class_id FK -> gym_class, booked_on,
        PRIMARY KEY(member_id, class_id))
```

3. **Test with rows.** "How many places are left in class 1?" → `capacity` minus the count of `booking` rows with `class_id = 1`. If the rows answer every Step-2 question, the design is sound (the full rows live in samples Lesson 18).

KEY IDEA: the method is a loop, not a straight line. Testing with rows (step 13) often sends you back to step 3. That is normal and healthy.

Part 10 — A worked example, end to end: a small library

Follow the merged method on a real little problem. This is the full end-to-end design: requirements → entities → attributes → keys → relationships → resolve hard patterns → normalise → final tables → sample rows → questions answered.

1. Purpose. Track which members borrow which copies of which books, and when they are due back.

2. Questions/actions. The list from Part 1, repeated so we can tick it off at the end:

#	Question / action
1	Who has "Clean Code" out right now?
2	Which loans are overdue?
3	How many loans has Ada had, ever?
4	Add a member; lend a copy; record a return.

3. Entities. Nouns: **member**, **book**, **copy**, **loan**. (A *book* is the title; a *copy* is one physical item of it — the library owns three copies of one book.) Note that *loan* is a relationship that earned its own table, because it has its own facts (the dates).

4. Attributes. member: given/family name, email, joined date. book: title, author, isbn. copy: shelf code, acquired date. loan: borrowed/due/returned dates. `returned_on` is deliberately allowed to be `NULL` — a `NULL` there *means* "still out".

5. Keys. Surrogate PK on each (`member_id`, `book_id`, `copy_id`, `loan_id`); `email` and `isbn` kept `UNIQUE` as natural keys. (student-panel would use UUIDs here — same decision, Part 5's note.)

6. Relationships. One book has many copies; each copy is of one book (1:N). One member has many loans; each loan is one member's (1:N). One copy has many loans over time; each loan is of one copy (1:N). A loan therefore ties one member to one copy.

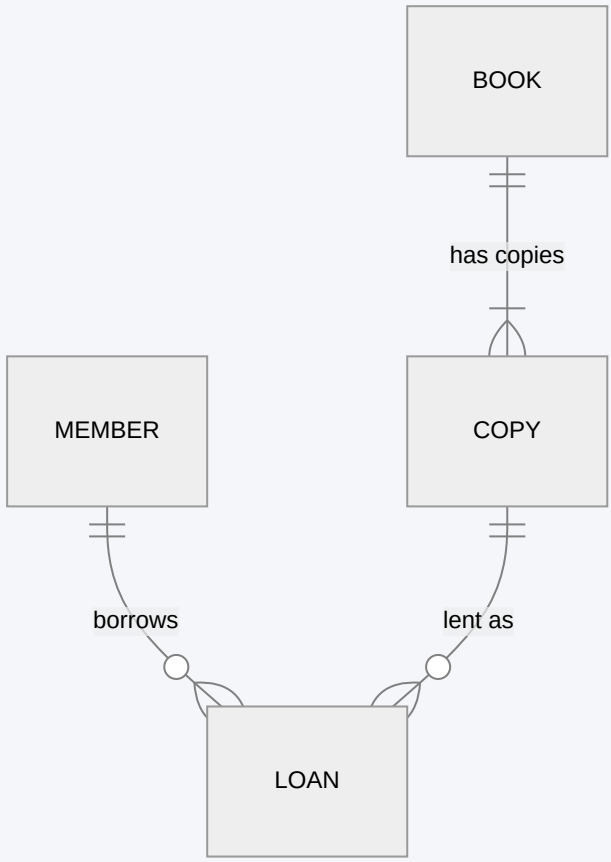
Pairing	both ways
book ↔ copy	one book has one-or-many copies; one copy is of exactly one book
member ↔ loan	one member has zero-or-many loans; one loan is of exactly one member
copy ↔ loan	one copy has zero-or-many loans over time; one loan is of exactly one copy

7. Hard patterns. None heavy here: "a member borrows many books" *looks* M:N, but the **loan** is the bridge that resolves it, and it carries its own facts (the dates) — the "one move" in action. (If a loan carried *no* facts at all, it would still need to be a table, because member↔copy over time is M:N.)

8–9. FKs and delete rules. `copy.book_id` → book (RESTRICT: don't delete a book that still has copies). `loan.copy_id` → copy and `loan.member_id` → member (RESTRICT: keep loan history).

FK	points at	on delete	said aloud
copy.book_id	book	RESTRICT	"you may not delete a book that still has copies"
loan.copy_id	copy	RESTRICT	"you may not delete a copy that still has loans"
loan.member_id	member	RESTRICT	"you may not delete a member who still has loans"

10. Diagram.



11–12. Normalise and map to tables. Each fact sits once; no repeating columns; `returned_on` is `NULL` until the item comes back (a meaningful NULL):

```
member(member_id PK, given_name, family_name, email UNIQUE, joined_on)
book(book_id PK, title, author, isbn UNIQUE)
copy(copy_id PK, book_id FK -> book, shelf_code, acquired_on)
loan(loan_id PK, copy_id FK -> copy, member_id FK -> member,
     borrowed_on, due_on, returned_on NULL)
```

13. Test with rows. Here is a consistent little dataset. The same rows appear in the samples file, so the two files tell one story.

`member`:

member_id	given_name	family_name	email	joined_on
1	Ada	Lovelace	ada@example.com	2024-01-10
2	Alan	Turing	alan@example.com	2024-02-02
3	Grace	Hopper	grace@example.com	2024-02-15

`book`:

book_id	title	author	isbn
1	The Pragmatic Programmer	Hunt & Thomas	9780135957059
2	Clean Code	R. C. Martin	9780132350884
3	Structure and Interpretation	Abelson & Sussman	9780262510875

`copy` (FK `book_id`):

copy_id	book_id	shelf_code	acquired_on
1	1	A-12	2023-11-01
2	1	A-13	2023-11-01
3	2	B-04	2024-01-20
4	3	C-09	2024-02-10

`loan` (FK `copy_id`, `member_id`; `returned_on` NULL = still out):

loan_id	copy_id	member_id	borrowed_on	due_on	returned_on
1	1	1	2024-03-01	2024-03-15	2024-03-10
2	3	1	2024-03-20	2024-04-03	*(null)*
3	2	2	2024-03-22	2024-04-05	*(null)*
4	4	3	2024-02-20	2024-03-05	2024-03-01

Answering the Step-2 questions against these exact rows (take "today" to be `2024-04-10`):

- **Q1 — Who has "Clean Code" out now?** "Clean Code" is `book_id = 2`; its only copy is `copy_id = 3`. The open loan of copy 3 is loan 2 (`returned_on` is NULL), whose `member_id = 1` → **Ada Lovelace** has it. Path: book → copy → loan (open) → member.
- **Q2 — Which loans are overdue?** Loans with `returned_on IS NULL` *and* `due_on < today`: loan 2 (due 2024-04-03) and loan 3 (due 2024-04-05). Loans 1 and 4 came back, so they are not overdue → **loans 2 and 3**.
- **Q3 — How many loans has Ada had ever?** Count `loan` rows with `member_id = 1`: loans 1 and 2 → **2 loans** (returned ones still count).

- **Q4 — The actions.** *Add a member:* insert `(4, Edsger, Dijkstra, edsger@example.com, 2024-03-01)` into `member`. *Lend a copy:* copy 1 is free (its last loan, loan 1, was returned), so insert a new `loan` row pointing at `copy_id = 1` and a `member_id`, with `returned_on` NULL. *Record a return:* set `returned_on` on the open loan (e.g. loan 3 gets `returned_on = 2024-04-06`), which also makes copy 2 free again.

Every Step-2 question is answerable from the rows, so the design is sound. Notice the design never stores "is this copy available?" — that is **derived**: a copy is available when it has no loan with `returned_on IS NULL`.

Part 11 — Design judgement (the choices with no single right answer)

Rules take you far; judgement takes you the rest of the way. The recurring calls:

- **Derive vs store** — recompute what you can; freeze what records a moment (Part 3). Denormalise only on purpose, for a measured reason.
- **The meaning of NULL** — decide, per column, whether blank is legal and what it *means*. `returned_on IS NULL` = "still out". Forbid NULL (`NOT NULL`) where blank would be nonsense.
- **Soft vs hard delete** — a **hard** delete removes the row for good; a **soft** delete keeps it but marks it (an `archived_at` date or `is_active` flag) so attached history survives. Choose by whether you ever need the past back. **student-panel soft-deletes / archives** so past scores stay meaningful.
- **A flexible/JSON column vs real columns** — a single "extra data" blob is tempting but cannot be constrained, validated, or queried like real columns. Prefer real columns for anything you will search or guard; reserve flex columns for genuinely open-ended extras.
- **Naming** — be consistent: singular table names or plural, `snake_case`, `*_id` for keys. Avoid reserved words (`order`, `group`, `user`). Consistency beats cleverness.

The meaning of NULL, shown. The same blank means different things in different columns, so decide each one:

Column	NULL allowed?	What NULL *means*
`loan.returned_on`	yes	the item is still out
`category.parent_id`	yes	this row is a root (no parent)
`member.email`	no (`NOT NULL`)	would be nonsense — a member must be contactable
`student.group_id`	no (`NOT NULL`)	every student must belong to a group

Soft vs hard delete, shown. A hard delete would erase Ada's row and (under RESTRICT) be blocked while loans exist, or (under CASCADE) take her loan history with her. A soft delete keeps the row and stamps it, so her past loans still make sense:

`member` with a soft-delete stamp (`archived_at` NULL = active):

member_id	full_name	archived_at
1	Ada Lovelace	*(null)*
2	Alan Turing	2024-05-01

Alan is archived — hidden from the active list, but his historical loans still point at a row that exists. This is student-panel's approach: `RESTRICT` plus an `archived_at` flag, so a scored period's history is never truly lost.

JSON blob vs real columns, shown. Tempting but weak:

student_id	full_name	extra
1	Ada	{"group":"Morning","phone":"555-1111"}

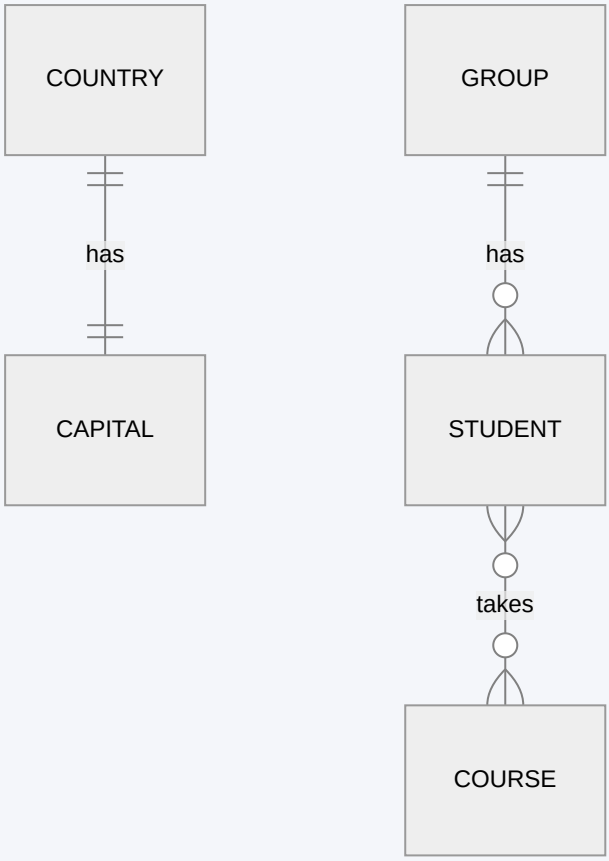
You cannot put a foreign key, a `UNIQUE`, or a `NOT NULL` on something inside that blob, and "list students in the Morning group" becomes string-fishing. Promote anything you will search or guard to a real column (`group_id`, a `phone` table); keep a flex column only for genuinely open-ended extras.

Part 12 — How this maps to student-panel (and where your way may differ)

As we re-derive student-panel together, you will recognise these shapes:

- **Surrogate keys** → UUID PKs + a unique `student_code` (Part 5's INT-vs-UUID note).
- **The bridge / "one move"** → the scores/evaluations table in the middle.
- **Self-reference trees** → groups and parameters, each with `parent_id`.
- **Frozen facts** → a recorded score stands for its period.
- **Soft delete** → archive instead of erase.

The shapes together, at the **principle level only** (the exact columns must be confirmed against the live database — see the note below):



0

Read it: a **group** is a tree (self-reference), a **parameter** is a tree, a **student** belongs to a group, and the central **score/evaluation** table is the bridge in the middle — one student × one parameter × one period, carrying the number as a **frozen fact**. That central bridge is "the one move" of Part 7, and it is the heart of the whole application.

Our standing rule: we do **not** change the student-panel project. You build your own version in `student-panel-I`, and wherever your way differs from student-panel's way, I will say so. Two differences are already visible: ids (you may use INT while learning; student-panel uses UUID) and deletes (you may hard-delete while practising; student-panel soft-deletes).

One thing to confirm against the real database. The exact keys and columns of student-panel's own tables should always be checked against the live schema before you rely on them — do not trust any lesson (including this one) for the precise column names. We will inspect the real schema together when we get there.

Part 13 — Ten practice questions, with verified answers

These are a fresh set, written and checked for this edition (they replace the original notebook's ten questions, whose answer key I could not open to verify). Try each on paper **before** reading the answer.

Q1. Entity or attribute? In a tiny corner shop, is *colour* an entity or an attribute? In a large paint manufacturer? **A1.** Purpose decides. In the corner shop, `colour` is just a plain value on a product → an **attribute** (a column). In the paint manufacturer, colours have their own facts (code, base, supplier) and many products share them → an **entity** (its own `colour` table, referenced by FK). Same word, different role. *Why this is the answer:* the test from Part 2 is "does it have its own facts, or will many rows share it?" A shop that only ever writes "red" on a product keeps no facts about red, so it stays a value. A factory that records red's pigment code and supplier, reused across hundreds of products, is keeping facts about red — and that is exactly what earns a table.

Q2. Type vs instance. "Student" — one thing or many? **A2.** `student` the **table** is the type (the kind). Each enrolled person is a **row** (an instance). You design the type; the rows arrive later. *Why:* you draw one cookie-cutter (the table, with its columns and keys), and the world stamps out many cookies (the rows). Designing is all about the cutter.

Q3. Many phone numbers. A member may have several phones. Give the design, and say why not `phone1, phone2, phone3`. **A3.** A member *has many* phones (1:N), so phones get their **own table** with the FK on the many side: `member(member_id PK, full_name)` and `phone(phone_id PK, member_id FK -> member, number)`. `phone1/2/3` caps the count and breaks 1NF; queries like "who has number X?" would have to name every column. *Why:* "many of something" is always a child table, never repeating columns. Three columns silently decide a member can never have a fourth phone, and the question "who owns 555-2222?" would have to search `phone1 OR phone2 OR phone3`. One child row per phone removes both problems and lets you count phones trivially.

Q4. Cardinality both ways. A group has many students; a student is in one group. State the relationship with numbers *and* participation, both ways. **A4.** "One group has **zero or many** students" (a brand-new group may have none → optional). "One student has **exactly one** group" (mandatory). That is 1:N, student side mandatory, group side optional. *Why:* you must say both directions, each with a number and an optionality. The group side is optional (a freshly created group can be empty), while the student side is mandatory (we do not allow a student with no group — that is a `NOT NULL` on `student.group_id`). As a diagram: `GROUP ||--o{ STUDENT`.

Q5. FK placement. For Q4, which table carries `group_id`? **A5.** The **student** table (the many side). One group → many students, so the pointer lives on each student row, not on the group. *Why:* put the FK where there can be *many* for one of the other. If `group_id` lived on `group`, one group could point at only one student — the opposite of what we want. On `student`, many rows can share `group_id = 1`, which is exactly "one group, many students".

Q6. M:N with a fact. Students take many courses; a course has many students; each pairing has a *grade*. Design it. Where does `grade` go, and what is the key? **A6.** M:N → a table in the middle. The grade is a fact *about the pairing*, so it lives on the bridge: `student(student_id PK, ...)`, `course(course_id PK, ...)`, `enrolment(student_id FK, course_id FK, grade, PRIMARY KEY(student_id, course_id))`. The composite `(student_id, course_id)` is the key — it also stops the same student being enrolled in the same course twice. **[correction applied]** If you instead gave `enrolment` its own surrogate id, you would still add `UNIQUE(student_id, course_id)` on purpose — unless the real world allows a repeat (e.g. retaking a course in a later term, in which case a term column joins the key). *Why:* a grade is not a fact about the student alone (she has different grades in different courses) nor about the course alone (different students score differently) — it is a fact about the *pairing*, so it belongs on the bridge row that represents that pairing. The composite key doubles as the "no duplicate enrolment" guard for free.

Q7. Self-reference tree. Categories can sit inside categories. Give the design and say what `parent_id IS NULL` means. **A7.** One table pointing at itself: `category(category_id PK, name, parent_id FK -> category NULL)`. `parent_id IS NULL` marks a **top-level** category (a root with no parent). Do **not** use `parent, grandparent, ...` columns — that caps the depth. *Why:* a single self-referencing column handles a tree of any depth — one level or ten — because each row only has to name its *immediate* parent. The root is the one row with nothing above it, which is precisely what a NULL `parent_id` records. (student-panel uses this exact shape for both its group tree and its parameter tree.)

Q8. Which normal form is broken? A table is `loan(loan_id PK, member_id, member_name, member_email, book_title)`. What is wrong, which rule names it, and what is the fix? **A8.** `member_name` and `member_email` depend on `member_id`, not on the whole key `loan_id` — a fact about the member has leaked into the loan row (update anomaly: change an email in one loan, the others still lie). That violates **3NF** ("nothing but the key"). Fix: move member facts to a `member` table and keep only `member_id` (FK) on `loan`. *Why 3NF and not 2NF:* the key here is the single column `loan_id`, not a composite, so there is no "part of the key" to depend on — 2NF cannot be the one broken. The problem is a non-key column (`member_name`) depending on *another non-key column* (`member_id`), which is the 3NF ("nothing but the key") violation. The leaked member facts are a drifting copy (contrast with a frozen fact, which would be fine), so moving them to `member` is correct.

Q9. Store it or derive it? (a) An order's *total* = sum of its lines. (b) The *unit price at the moment of sale*. Store which, and why? **A9.** (a) **Derive** the total — it is recomputable from the lines, so storing it risks drift. (b) **Store** `unit_price_at_sale` **[correction applied]** — it is a *frozen historical fact*: the price the customer actually paid, which must not change when the catalogue price changes later. "Derive, don't store" applies to values you can always recompute; a captured point-in-time value is the exception. *Why the two differ:* the total is a pure function of data that is *already* stored (the lines) and never needs to disagree with them — so recompute it and avoid the risk of a stale copy. The sale price, by contrast, is captured *against* a source that will move (the catalogue price); if you did not freeze it, next month's price change would silently rewrite history. The test from Lesson

16: "if the source changes, must this value change too?" Total: yes → derive. Sale price: no → freeze.

Q10. Read the diagram aloud. What does `member ||--o{ loan` say, in both directions?
A10. Left→right: "one member has **zero or many** loans" (`||` = exactly one on the member end; `o{` = zero-or-many on the loan end). Right→left: "each loan belongs to **exactly one** member." A new member with no loans is allowed; a loan with no member is not. *Why:* read each end outside-in. On the member end, `||` is min-one/max-one = "exactly one". On the loan end, `o{` is min-zero/max-many = "zero or many". The zero is what lets a brand-new member exist with no loans yet; the mandatory "exactly one" on the member end is what forbids an orphan loan.

A self-check rubric. Before you call any design finished, run it through this list. If you can tick every row *with a reason*, you have done the work; any row you cannot tick is exactly the thing to bring to the next conversation.

Ask yourself	A pass looks like	Re-read
Does every cell hold **one** value (no "A, B" lists)?	atomic cells	Part 8 (1NF)
Any repeating columns (`phone1, phone2, …`)?	none	Part 8 (1NF)
Does each table have a **primary key**?	one per table	Part 5
Does every **M:N** have a bridge table?	yes	Part 7
Does each fact live in exactly **one** place?	yes	Part 8 (2NF/3NF)
Is each **FK on the many side**, pointing at a row that really exists?	yes	Parts 5–6
Did you choose an **ON DELETE** rule for every FK, said as a sentence?	yes, per FK	Parts 5, 11
Any stored value that could **contradict** what it is derived from?	none (or a frozen fact)	Parts 3, 8
Did you say each relationship **aloud, both ways** (number + optionality)?	yes	Part 4
Did you note every decision you were **unsure** about?	yes — bring them	(all)

Glossary

- **Anomaly (insert / update / delete)** — the three symptoms of redundancy: you cannot record a thing (insert), copies drift apart (update), or removing one row destroys an unrelated fact (delete). Normalisation removes them.
- **Attribute** — a property of an entity; becomes a **column**.
- **Bridge (junction / associative) table** — the table in the middle of a many-to-many, carrying one FK to each side and any facts about the pairing.

- **Candidate key** — any column(s) that could uniquely identify a row.
- **Cardinality** — how many on each side of a relationship (1:1, 1:N, M:N).
- **Composite key** — a key made of more than one column.
- **Conceptual / logical / physical** — the three design levels: ideas → tables & keys → the actual SQL types and constraints in a specific engine.
- **Crow's-foot** — the diagram notation: `|` one, `o` zero/optional, `{` many.
- **Denormalisation** — storing a redundant value on purpose, for speed, as a conscious trade with a known upkeep cost.
- **Derived value** — a value computable from others; prefer not to store it.
- **Entity** — a kind of thing worth its own table; a **type**. A single one is an **instance** (a **row**).
- **Foreign key (FK)** — a column pointing at another table's primary key; it sits on the **many** side.
- **Frozen fact** — a value captured on purpose at a moment in time (e.g. `unit_price_at_sale`); kept, not re-derived.
- **Multi-valued attribute** — a fact with several values for one row (a member's phones); becomes its own table, never repeating columns or a list in a cell.
- **Normalisation (1NF/2NF/3NF)** — removing redundancy so each fact lives once: "the key, the whole key, and nothing but the key."
- **NULL** — "no value here"; on an FK it means the optional link is absent.
- **ON DELETE RESTRICT / CASCADE / SET NULL** — what happens to children when a parent row is deleted: block it / delete them too / blank their FK.
- **Participation** — whether a side is mandatory (min one) or optional (min zero).
- **Primary key (PK)** — the one chosen candidate key that identifies each row.
- **Referential integrity** — the rule that every FK must point at a row that exists (no orphans).
- **Relationship** — a verb linking entities ("a member *borrows* a copy").
- **Self-reference** — a table with a `parent_id` FK to itself, modelling a tree; `NULL` at the root.
- **Soft delete** — marking a row as gone (an `archived_at` date / `is_active` flag) instead of erasing it, so attached history survives.
- **Surrogate key** — a system-made id with no business meaning: an auto-number **INT**, or a **UUID** (student-panel uses a UUID PK plus a unique human code).
- **Natural key** — a key made of real-world data (e.g. an ISBN).
- **Ternary relationship** — a fact that joins three entities at once; modelled as a middle table holding three FKs.
- **Weak entity** — one that cannot be identified without its parent.

End of the revised lesson. Companion files: the samples/examples live in `database-design-revised-samples.md`; the hands-on practice in `database-design-practice-lesson.txt`; the concept bridge in `database-design-companion-lesson.txt`; and the review of the other-AI pieces in `database-design-AI-pieces-review.txt`.